Research on Quantitative Stock Selection and Effect Evaluation Based on Machine Learning

Ziwei Ai, Wanni Hu, Huixuan Meng

Beijing Normal University - Hong Kong Baptist University United International College

Python for Finance (1001)

Dr. Eric Pengfei ZHAO

December 2024

Table of Contents

1 Introduction

1.1 Background

Quantitative investment originated in the United States, with the world's first convertible bond arbitrage quantitative investment fund established in 1969 by Thorpe and Reagan, marking the beginning of quantitative investing. During the 1970s and 1980s, the establishment of the Chicago Board Options Exchange in 1973 led to the rapid acceptance of option pricing theory on Wall Street, which further accelerated the development of quantitative investing. In 1979, the world's first actively managed quantitative fund was launched by Barclays Global Investors. In 1988, Simons founded the Renaissance Technologies Medallion Fund, achieving an annualized return of 70%, which further propelled the boom in quantitative investing. In the 1990s, several related investment theories, such as Modern Portfolio Theory and CAPM (Sharpe, 1964), were discovered, promoting the development of quantitative investing. In the early 21st century, the bursting of the internet bubble further contributed to the expansion of quantitative hedge fund sizes. By 2011, the global assets under management (AUM) of quantitative investment funds had reached $2.4 trillion, and by 2017, it had surpassed $3 trillion.

In contrast, quantitative investing started relatively late in China. The first quantitative fund in China was launched in 2004. Then, in 2010, the listing of the CSI 300 futures marked the entry of quantitative investing into China's 1.0 era. Following this, the launch of the CSI 500 index futures in 2015 signaled the arrival of the 2.0 era of quantitative investing. In 2019, the establishment of the China Securities Regulatory Commission (CSRC) marked the beginning of the 3.0 era of China's quantitative economy.

A large number of theories in quantitative investing come from mathematical finance. It is a tool for transforming securities investment methods into models through the application of finance, statistics, mathematics, and computer science, with the aim of achieving high absolute returns or excess returns through high-probability success (Ding, 2014).

The essence of quantitative investing is stock price prediction; however, numerous factors influence stock prices, and their relationships are complex and non-linear. With the growing development of artificial intelligence, particularly its branch, machine learning algorithms, which automatically search for patterns and relationships in data and make predictions about unknown data, many studies have shown that linear machine learning models outperform

linear regression models in quantitative investing. Moreover, non-linear machine learning methods have shown superior predictive abilities compared to linear machine learning algorithms (Li, Shao, & Li, 2019). Therefore, this has contributed to this study's exploration of how machine learning, when applied to quantitative investing, aids in stock return prediction and the development of quantitative investing.

1.2 Research Significance and Purpose

Since machine learning was established as an independent discipline in 1980, it has gradually been widely applied across various fields, with quantitative investing being no exception. However, many studies have found that, despite the significant advantages of machine learning models in capturing relationships within financial data and predicting stock returns, there is no conclusive agreement on the accuracy of their predictive performance. For example, an algorithm that performs well on one dataset does not necessarily outperform others on a different dataset.

This study uses data from all constituent stocks of the CSI 500 index from August 16, 2024, to November 29, 2024, as the research subject. It compares the predictive performance of four machine learning models in forecasting stock price movements and applies the trained machine learning models to the construction and backtesting of quantitative strategies.

1.3 Research Framework

This empirical study is divided into five main sections: data collection, data preprocessing, feature dimensionality reduction, model prediction and evaluation, and quantitative strategy construction and backtesting.

Data Collection

First, the study collects data from the Wind platform for all constituent stocks of the CSI 500 index from August 16, 2024, to November 29, 2024. Fifteen indicators are gathered (8 financial indicators and 7 technical indicators), and the future weekly return direction (up or down) is used to label the data.

Data Preprocessing

The constituent stocks of the CSI 500 index are cleaned and filtered, removing stocks that may affect the subsequent strategy analysis, such as ST stocks, ChiNext stocks, stocks listed for less than two years, and stocks that have been suspended.

Feature Dimensionality Reduction

Principal Component Analysis (PCA) is applied to reduce the dimensionality of the aforementioned 15 indicators (8 financial and 7 technical indicators). The top 8 key principal components are selected based on their scores, and the original data is aligned with these principal components.

Model Prediction and Evaluation

The reduced-dimensional data is used to train four machine learning models. Techniques such as grid search are applied for parameter tuning. Multiple evaluation metrics are then used to assess the performance of the trained models.

Quantitative Strategy Construction and Backtesting

The trained machine learning models are used to construct quantitative strategies. Data from November 1, 2024, to November 29, 2024, is used for strategy backtesting. The results are analyzed and compared to evaluate the profitability of each model.

2 Data Acquisition

2.1 Time Selection

This study selected data from all constituent stocks of the CSI 500 index for the period from August 16, 2024, to November 29, 2024, and divided the data into two usage phases:

From August 16 to November 1: This phase is used for parameter setting, data preprocessing, and model training.

From November 1 to November 29: During this phase, predictions are made by applying the trained models to forecast the stock returns for the upcoming week. The prediction accuracy and backtesting results are verified using data from this period.

2.2 Stock Picking

Remove ST Stocks: Delete stocks whose names contain "ST," as ST stocks usually carry financial risks that may affect subsequent strategy analysis.

Remove GEM Stocks: Delete stocks whose codes begin with "30," as these are from the ChiNext board, which typically experiences higher volatility and greater risks.

Remove Stocks that Have Been on the Market for Less than Two Years: Delete stocks that have been listed for less than two years, as these stocks tend to have less data and are more unstable.

Remove Suspended Stocks: Delete all suspended stocks, as they cannot be traded in real time and cannot provide effective data for strategy analysis. (1 minute)

2.3 Factor Selection

In most quantitative investment studies, constructing effective investment strategies typically relies on multiple indicators, which can be broadly categorized into two types: technical indicators and financial indicators. Stock prices inherently reflect a company's intrinsic value, while financial indicators can effectively reveal a company's operational status and financial health. Technical indicators, on the other hand, reflect changes in market sentiment and stock price trends, and are widely used in quantitative investing. This study selects the following two types of indicators for factor analysis:

2.3.1 Financial Data

Total Market Value

The total market value is the company's stock price times its circulating shares; it reflects the company's size relative to the market. It is also a significant measure to evaluate a company's market size and power. Generally, bigger market capitalization of companies indicates more stability and risk resistance. Leading companies in terms of market cap lead in market performance, experience better funding and investment value.

Price-to-Earnings Ratio (PE)

Price-to-earnings ratio is the ratio of a company's market value to its earnings per share (EPS). it is used to measure the stock price against the company's profitability. It also mirrors the level of the stock's valuation. A lower PE can mean that the stock is undervalued and therefore a good investment, whereas a higher PE can mean that the market is expecting a high growth rate for the company in the future.

Price-to-Book Ratio (PB)

The price-to-book ratio is the ratio of a company's market value to its book value, and helps determine if a stock is either undervalued or overvalued. It is also an indication of the company's book value compared to its market price. Generally, stocks with a lower PB are

seen as undervalued and possible investment opportunities. A High PB may show that stock is overvalued, or that the market has sufficient high expectations for its future potential.

Earnings Per Share (EPS)

An EPS, or earnings per share, represents the company's net profit divided by the number of shares outstanding. It is a rudimentary measure of a company’s profitability, and a higher EPS generally indicates better earning capacity. EPS has a direct correlation to profitability, which is why many investors gravitate towards companies with a high EPS — they often assume that such companies are positioned for growth.

Return on Equity (ROE)

Return on equity indicates the ratio of net profit of the company to its shareholders' equity, in order to assess the return level of shareholders' investments. This is an essential measure of a corporation’s earnings, which considers its efficiency in effectively using shareholder funds. Higher ROE usually suggests that a business's profitability utilizing shareholder funds is greater, which reflects better competitiveness.

Net Profit

Net profit (also called net income or net earnings) is what’s left after a company’s total revenue the amount of money it takes in — less its total costs the company’s expenses, including recurring ones like salaries, rent, and raw materials. It is an immediate reflection of the operational efficiency of a company and signifies the period for which a company is profitable. The net profit is a strong indicator that the business is profitable and able to compete in its market.

Operating Revenue

Operating revenue is the revenue generated from a company's normal business operations as opposed to revenue generated from other sources. It represents the primary revenue generator for a company's fundamental operations, and a strong indicator of competitiveness and growth potential in an industry. Going, going, gone Stable operating revenue typically means business growth is healthy.

Total Asset Turnover

Total asset turnover is a ratio of a company’s revenue to its total assets, and it shows how efficiently a company is at generating an income from its assets. This is a key measure of efficient asset utilization. A higher asset turnover means that a firm is more efficient in using assets to generate revenue.

2.3.2 Technical Indicator Data

Moving Average Convergence/Divergence (MACD)

This shows the difference between the short-term and long-term exponential moving averages, which depicts the strength, direction, and possible reversal signals in market trends. It is one of the most extensively employed techniques that helps in identifying the time point of trend reversal and rate of change in price moments. MACD is of great use for investors because it helps them to realize overbought or oversold market conditions.

RSI

The RSI takes the magnitude of recent gains relative to recent losses to compute the relative strength of a stock, hence reflecting its price momentum. It is an important indicator for the assessment of overbought or oversold conditions of stocks. Values of RSI above 70 are considered overbought, while values below 30 are considered oversold, providing value for market forecasting.

On-Balance Volume

On-balance volume cumulates volumes on advancing days and liquidates volumes on declining days to measure the money inflow or outflow of an instrument in the market. OBV can reflect both fund flow and trend strength better by setting price and volume change relations. It may confirm some stock price movement through inflow and outflow of capital.

Commodity Channel Index

The CCI shows the difference of a stock price from its average value and thus assists in finding overbought or oversold market conditions, which may lead to market reversals. It measures price deviation relative to the average, often used for assessing the strength of trends and reversal points. A CCI greater than 100 means overbought conditions, while less than -100 means oversold conditions, hence providing considerable information for decision-making.

Average True Range (ATR)

ATR reflects the range of price fluctuations in a certain period, characterizing market volatility and the possible amplitude of the movement. It is a volatility indicator and a tool for assessing risks. The greater the value of ATR, the larger the market volatility and risks are.

Williams %R (WR)

Williams %R reflects the comparison of price movements for some fixed period by calculating a relative position of the price for that period. It reflects buying and selling pressures in the market. WR is used for the assessment of whether a stock is overbought or oversold. This indicator is often applied for short-term trading.

The Momentum Index characterizes the speed of changes in prices and has a forecasting character, providing insight into future tendencies of share prices, especially in strong-trend markets. It signals the rate and direction in which the price is moving.

2.4 Label Each Stock

The future weekly return direction for each stock is labeled. If the return for the upcoming week is positive, the label is 1, indicating an upward prediction; if the return is negative, the label is 0, indicating a downward prediction. This method transforms the data into a format suitable for classification prediction using machine learning models.

2.5 Model Assumption

Each row of data (8 financial indicators and 7 technical indicators) in the "model_data" is independent and does not affect each other.

3 Data Preprocessing

Handling Missing Values (Mean Imputation)

This study organizes 16 groups of weekly data for the CSI 500 constituent stocks within the time period into a complete dataframe, and processes the 15 indicators (including 8 financial indicators and 7 technical indicators) to ensure the continuity and completeness of the data. For missing values, this study uses mean imputation to replace the missing data points. Specifically, for each factor's time series, any missing value is replaced with the mean of that factor's column to ensure the usability of each factor.

4 Data Dimensionality Reduction - PCA

4.1 PCA Principle Introduction

Principal Component Analysis (PCA) is a common dimensionality reduction technique that effectively reduces the dimensionality of data while retaining the main feature information as much as possible. It is used to predict the next week's price movement based on the current week's related data. In this study, PCA is applied to reduce the 15 indicators to lower

dimensions, thus improving the efficiency and effectiveness of subsequent model training (Uçkan, 2024).

Data Standardization

To ensure that all features have the same scale and avoid the impact of indicators with different units on the analysis results, Z-score standardization is applied to all data. The standardization formula is:

$$z = \frac{x - \mu}{\sigma}$$

Where:

x is the original data,

μ is the mean of the feature,

σ is the standard deviation of the feature,

Calculate the Covariance Matrix.

The covariance matrix is calculated to describe the correlations between feature variables. The covariance matrix $\sum$ represents the linear relationships between the features in the dataset. The covariance matrix can be calculated using the following formula:

$$\sum = \frac{1}{n-1} X^T X$$

Where:

X is the data matrix,

X^T is the transpose of the data matrix,

$\sum$ is a $p \times p$ matrix, each element $\sum_{ij}$ of the covariance matrix represents the covariance between i and j features, reflecting their linear relationship.

The Pearson correlation coefficient is a commonly used statistic to measure the linear relationship between two variables, with values ranging from [−1,1], where 1 indicates a perfect positive correlation, -1 indicates a perfect negative correlation, and 0 indicates no correlation.

Eigenvalues and Eigenvectors

By performing eigenvalue decomposition on the covariance matrix, we can obtain the eigenvalues and their corresponding eigenvectors. The eigenvalues represent the variance of the principal components in the data, while the eigenvectors represent the direction of the principal components. The formula to calculate the eigenvalues and eigenvectors is:

$$\sum v = \lambda v$$

Where:

λ is the eigenvalue,

v is the corresponding eigenvector.

Calculate the Principal Component Scores.

The score of each sample on the selected principal components is calculated. The principal component score matrix is given by the following formula:

$$Z = XV_m$$

Where:

X is n × p data matrix,

n is the number of samples,

p is the number of original features,

V_m is the matrix of eigenvectors containing mmm principal components, where each column is an eigenvector representing the direction of a principal component,

Z is the principal component score matrix calculated, with dimensions n × m, where each column represents the score of each sample on the corresponding principal component

Select Principal Components

The key to principal component analysis is selecting the eigenvectors corresponding to the largest eigenvalues as the principal component directions. These eigenvectors form a new feature space, where each principal component corresponds to the direction of the largest variance in the data. Then, the appropriate number of principal components is selected based on the cumulative variance contribution rate. Generally, a cumulative variance contribution rate of 85% or greater is used as the standard. This means that the selected principal components can explain at least 85% of the variance in the original data.

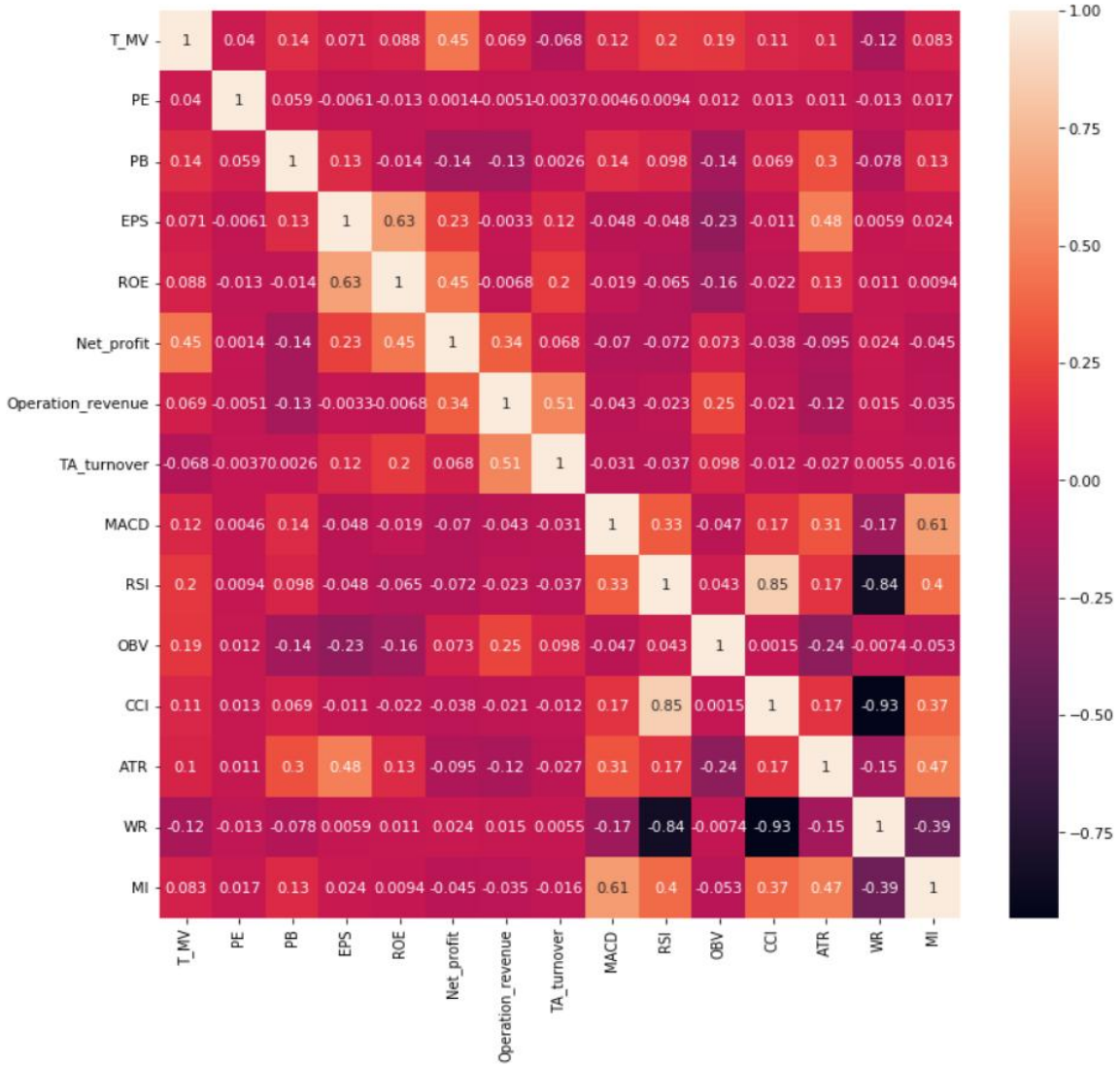
4.2 Dimensionality Reduction

4.2.1 Data Preparation

First, data for 15 selected indicators (8 financial indicators and 7 technical indicators) of the CSI 500 constituent stocks from August 16, 2024, to November 29, 2024, are extracted from the Wind database. These data will serve as the input dataset for PCA. To ensure that all features are on the same scale and to avoid the impact of different units on the analysis results, Z-score standardization is applied to all data.

4.2.2 Feature Variable Correlation Analysis

In order to learn more about the relationships between features, a correlation analysis was done on standardized data before dimensionality reduction. Specifically, the Pearson correlation coefficient matrix between features was calculated using "X_s.corr(method='pearson')" for assessing the linear relationships among features. The correlation matrix was then visualized using a heatmap "sns.heatmap" for intuitively presenting the relationship of features. Each element of the matrix denotes the correlation coefficient between two corresponding features, and the color intensity of the plot reflects the magnitude of the correlation coefficient. From this heat map, one can easily see that lighter colors reflect a higher positive correlation and darker colors reflect a higher negative correlation, thus enabling an intuitive comparison between any two of the 15 indicators.



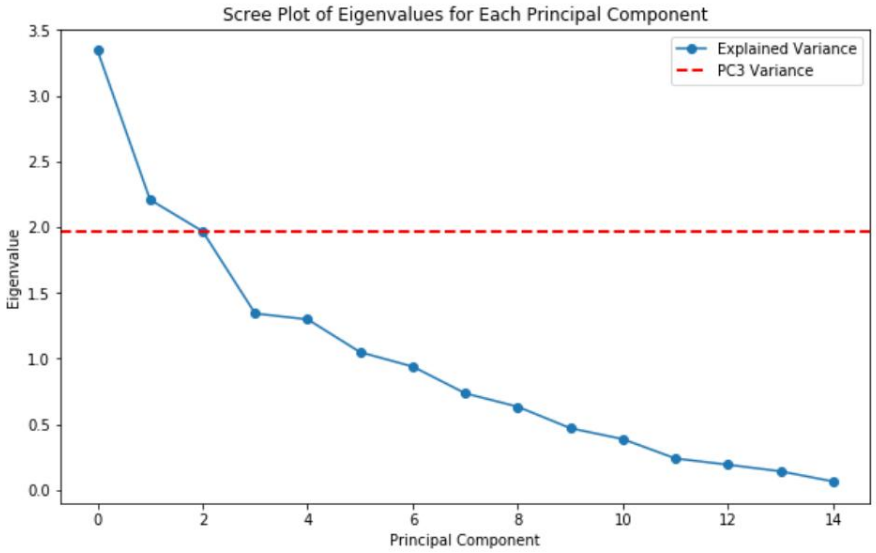
4.2.3 Principal Component Extraction

In the standardized dataset, principal component extraction is done by "model = PCA()" in Python. It does eigenvalue decomposition on the covariance matrix of data to get eigenvalues and eigenvectors for each principal component. Each eigenvector explains the direction of a principal component, while the corresponding eigenvalue explains the variance explained by that principal component. Using these kinds of eigenvectors, we project our original data onto a new feature space.

Eigenvalue and Variance Contribution Calculation

Now, let's determine eigenvalues ("model.explained_variance") and the contribution rates ("model.explained_variance_ratio_") of the principal components. Where eigenvalue will signify what percentage a principal component explained within the variation, while on the other side, it tells the rate of contribution against the overall variance explained by the principal component. Specifically, compute the eigenvalues of the covariance matrix that describe the variance explained by each principal component:. The contribution rate is calculated as the ratio of each eigenvalue to the sum of all the eigenvalues of the principal components: showing the amount of total variance that can be explained by the component.

To visually select the principal components, a scree plot (or "elbow plot") is commonly used to display the eigenvalues (or variance contribution ratios) of each principal component. This plot helps to determine which principal components significantly explain the variability in the data. In a scree plot, the principal components with larger eigenvalues appear on the left, while the principal components with progressively smaller eigenvalues are located on the right.



Principal Component Scores

The scores of the principal components are determined by the projection of the original data onto the selected principal components. More precisely, the multiplication of matrices between the original data and the eigenvectors of the principal components produces the coordinates of each sample in the space of principal components. These scores reflect the distribution of the data along the directions of the principal components and can be used for further analysis of the structure and trends of the data.

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	\
0	2.255651	-0.445994	-0.692624	-1.206116	0.767749	0.004496	0.134444	
1	2.686402	-0.401197	-0.576691	-1.231302	0.710576	0.014401	0.105584	
2	1.510078	-0.673452	-0.911395	-0.722860	0.477616	-0.102555	0.152594	
3	0.359981	-0.889297	-1.333569	-0.217473	0.257333	-0.173507	0.172665	
4	1.980533	-0.576333	-0.682854	-1.062978	0.590743	-0.003618	0.089417	

	PC8	PC9	PC10	PC11	PC12	PC13	PC14	\
0	1.913711	-0.794513	0.796080	0.303766	0.276787	-0.059327	-0.072013	
1	1.942632	-0.738963	0.891246	0.442817	0.343480	-0.104257	-0.566332	
2	1.972466	-0.766289	0.802578	0.294337	0.379115	-0.044003	-0.406585	
3	1.932981	-0.785572	0.840031	0.248980	0.424424	-0.093512	-0.618465	
4	1.984171	-0.797632	0.785107	0.273133	0.324608	-0.023661	-0.101218	

	PC15
0	0.069728
1	-0.110985
2	-0.294865
3	-0.267867
4	-0.210748

Using Cumulative Variance Contribution Rate to Select Principal Components

Based on the contribution rates and the accumulated contribution rate, a few numbers of principal components are chosen. The accumulative variance contribution rate stands for the contribution rate of the selected principal component to the total variation. In general, in order to ensure the validity of the data after reduction, principal components explaining at least 85% of the total variance are usually selected to make sure that the main information of the data is retained. According to the contribution rate in this study, the first 8 principal components are chosen for dimensionality reduction.

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
0	2.255651	-0.445994	-0.692624	-1.206116	0.767749	0.004496	0.134444	1.913711
1	2.686402	-0.401197	-0.576691	-1.231302	0.710576	0.014401	0.105584	1.942632
2	1.510078	-0.673452	-0.911395	-0.722860	0.477616	-0.102555	0.152594	1.972466
3	0.359981	-0.889297	-1.333569	-0.217473	0.257333	-0.173507	0.172665	1.932981
4	1.980533	-0.576333	-0.682854	-1.062978	0.590743	-0.003618	0.089417	1.984171
...	...	...	...	...	...	...	...	...
6971	0.540615	3.313938	1.179439	0.602597	-0.365906	-1.133744	1.111991	0.660225
6972	-0.956646	3.260871	0.911421	0.940791	-0.593417	-1.195402	1.138607	0.776270
6973	-1.065099	3.200108	0.851238	0.860896	-0.530948	-1.128144	1.081428	0.648842
6974	1.453335	3.549723	1.667940	0.075616	-0.301082	-0.978197	0.989409	0.839368
6975	0.616968	3.405363	1.271217	0.740199	-0.495597	-1.228784	1.188903	0.875164

6976 rows × 8 columns

4.2.4 Final Dataset

After the steps above, the final dataset obtained is a reduced-dimensional dataset containing the principal components extracted through PCA. This dataset will be used for subsequent machine learning model training and quantitative strategy development. Through PCA, the original 15 indicators have been compressed into 8 key principal components, reducing the feature dimension while retaining most of the data's information, significantly improving the efficiency of data processing.

	ts_code	Name	flag	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
0	000009.SZ	中国宝安	0	2.255651	-0.445994	-0.692624	-1.206116	0.767749	0.004496	0.134444	1.913711
1	000009.SZ	中国宝安	1	2.686402	-0.401197	-0.576691	-1.231302	0.710576	0.014401	0.105584	1.942632
2	000009.SZ	中国宝安	1	1.510078	-0.673452	-0.911395	-0.722860	0.477616	-0.102555	0.152594	1.972466
3	000009.SZ	中国宝安	0	0.359981	-0.889297	-1.333569	-0.217473	0.257333	-0.173507	0.172665	1.932981
4	000009.SZ	中国宝安	0	1.980533	-0.576333	-0.682854	-1.062978	0.590743	-0.003618	0.089417	1.984171
...	...	...	...	...	...	...	...	...	...	...	...
6971	689009.SH	九号公司-WD	1	0.540615	3.313938	1.179439	0.602597	-0.365906	-1.133744	1.111991	0.660225
6972	689009.SH	九号公司-WD	1	-0.956646	3.260871	0.911421	0.940791	-0.593417	-1.195402	1.138607	0.776270
6973	689009.SH	九号公司-WD	0	-1.065099	3.200108	0.851238	0.860896	-0.530948	-1.128144	1.081428	0.648842
6974	689009.SH	九号公司-WD	1	1.453335	3.549723	1.667940	0.075616	-0.301082	-0.978197	0.989409	0.839368
6975	689009.SH	九号公司-WD	1	0.616968	3.405363	1.271217	0.740199	-0.495597	-1.228784	1.188903	0.875164

6976 rows × 11 columns

5 Stock Price Prediction

There are many machine learning algorithms, which can be categorized based on task objectives into regression algorithms, clustering algorithms, and classification algorithms. According to the way the algorithm learns, they can be classified into supervised, semi-supervised, unsupervised, and reinforcement learning algorithms. Since this study involves labeling stock data for rise and fall predictions, only supervised learning binary classification algorithms are used. We used support vector machine (SVM), logistic regression, K-Nearest Neighbors(KNN), and random forest algorithms to predict stock price movements.

5.1 Training Set and Testing Set

In this study, to evaluate the model effectively and avoid overfitting, the data is split into a training set and a test set. The time period for the dataset is from August 16, 2024, to November 29, 2024. The specific division is as follows:

Training Set

Data from August 16, 2024, to November 1, 2024, used for model training.

Testing Set

Data from November 1, 2024, to November 29, 2024, used for model validation and performance evaluation.

5.2 Grid Search

In machine learning, when training a model, it is usually necessary to select hyperparameters that can impact the model's performance. To help the model better fit the data, this study uses the grid search method. This involves arranging and combining a set of possible parameter values and then using cross-validation to evaluate the performance of different parameter combinations, ultimately selecting the optimal combination.

For example, for the support vector machine model, we assume that the regularization parameter (C), which is used to adjust the tolerance for training error and control model complexity, has four possible values: 0.1, 1, 10, and 100. Additionally, the kernel type (kernel) and kernel coefficient (gamma) each have four and three possible values, respectively. Grid search will explore all combinations of these hyperparameters and return the best one.

```
# Define a grid of hyperparameters for SVM
param_grid_SVM = {
    'C': [0.1, 1, 10, 100],  # Regularization parameter
    'kernel': ['linear', 'rbf', 'poly'],  # Type of kernel function
    'gamma': ['scale', 'auto'], # Kernel coefficient 0.1, 1, 10
}
```

5.3 Evaluating indicator

After determining the hyperparameters, this study uses the following metrics to evaluate the performance of the four machine learning models.

Accuracy

This metric represents the percentage of correct predictions out of the total samples.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

TP (True Positive): The number of samples that were correctly predicted as positive class,

TN (True Negative): The number of samples that were correctly predicted as negative class,

FP (False Positive): The number of samples that were incorrectly predicted as positive class,

FN (False Negative): The number of samples that were incorrectly predicted as negative class.

Precision

This metric measures the proportion of actual positive samples among all samples predicted as positive.

$$Precision = \frac{TP}{TP + FP}$$

Recall

This metric measures the proportion of correctly predicted positive samples out of all actual positive samples.

$$Recall = \frac{TP}{TP + FN}$$

F1 Score

This metric is the harmonic mean of precision and recall, considering both precision and recall performance.

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

To avoid bias from using a single metric, this study will comprehensively consider the above metrics in evaluating the model.

5.4 Machine learning models

5.4.1 Support Vector Machines

Cortes and Vapnik (1995) introduced the Support Vector Machine (SVM), which is based on statistical learning theory and is a powerful learner. Its working principle is to find a hyperplane in the data distribution that serves as the decision boundary, minimizing classification errors on the data as much as possible.

The model determines the decision boundary by maximizing the margin between the sample points on either side of the hyperplane, transforming the problem into a convex quadratic programming problem for solving. In this study, since we assume that the price change in the next week is determined by eight main indicators, the sample size is eight-dimensional, and the hyperplane should be seven-dimensional.

This study considers three hyperparameters: the regularization parameter, the type of kernel function, and the coefficients of the kernel function.

```
# Define a grid of hyperparameters for SVM
param_grid_SVM = {
    'C': [0.1, 1, 10, 100],  # Regularization parameter
    'kernel': ['linear', 'rbf', 'poly'],  # Type of kernel function
    'gamma': ['scale', 'auto'], # Kernel coefficient 0.1, 1, 10
}
```

The regularization parameter (C) controls the degree of fit of the classifier to the training data. It adjusts the tolerance for training errors to control the complexity of the model. A larger C will more strictly fit the training data, while a smaller C will tolerate more classification errors, thus obtaining a larger margin.

SVM uses the kernel function type (kernel) to map the data from a low-dimensional space to a high-dimensional space, in order to find a linearly separable hyperplane in the high-dimensional space. This study assumes that the kernel function could be a linear kernel, radial basis function kernel (rbf), or polynomial kernel (poly), depending on whether the data is linearly separable and the complexity involved.

The coefficient of the kernel function (gamma) determines the influence range of a single training sample on the decision boundary. A higher gamma value means that the influence

range of each data point on the decision boundary is smaller, and the model may overfit; a lower gamma value means that the influence range of each data point is larger, and the model may underfit.

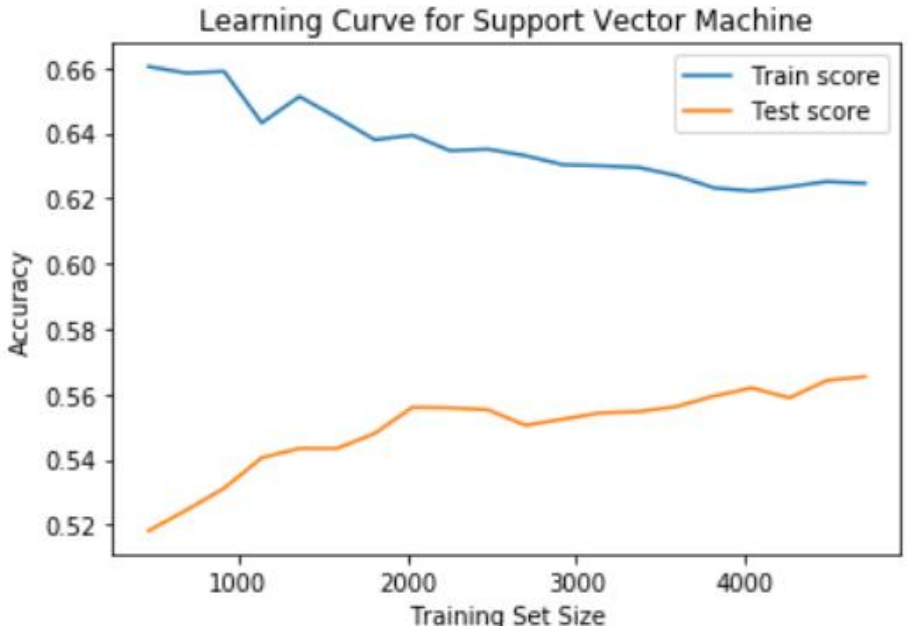
This study establishes two candidate values for gamma: “scale” and “auto”. “Scale” is defined as “$1/(n\_features * X.var())$”, where “$X.var()$” is the variance of the features. Auto sets gamma as “$1/n\_features$”, meaning each feature has an equal influence range.

Model fitting:

```
# Initialize the SVM classifier
svm = SVC(random_state=42)

# Perform hyperparameter tuning using GridSearchCV
grid_search_SVM = GridSearchCV(svm, param_grid_SVM, cv=10, n_jobs=-1, verbose=1)
grid_search_SVM.fit(X_train_SVM, y_train_SVM)

# Print the best parameters found
print("Best parameters found: ", grid_search_SVM.best_params_)

# Train the SVM model with the best parameters
best_svm = grid_search_SVM.best_estimator_

# Evaluate the model on the test set
y_pred_SVM = best_svm.predict(X_test_SVM)
```

Fitting result:

The optimal hyperparameter result is: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}, indicating that this study used a more complex, non-linearly separable dataset.



5.4.2 Logistic regression

Logistic regression is a linear classifier referred to as "regression." It essentially evolved from linear regression and is a generalized linear regression analysis model, falling under

supervised learning in machine learning. The core idea is to use a linear combination of input features and then map the result to a value between 0 and 1 using a Sigmoid function, thereby making a classification decision. Typically, the model predicts the sample to belong to class 1; if the mapped value is less than 0.5, the prediction is class 0.

This study considers regularization strength, regularization type, and optimization algorithm as the hyperparameter choices for this model.

```
# Grid search to find optimal hyperparameters
param_grid_lo = {
    'C': np.logspace(-3, 2, 6),
    'penalty': ['l1', 'l2', 'elasticnet'],
    'solver': ['lbfgs', 'liblinear', 'saga', 'newton-cg', 'sag']
}
```

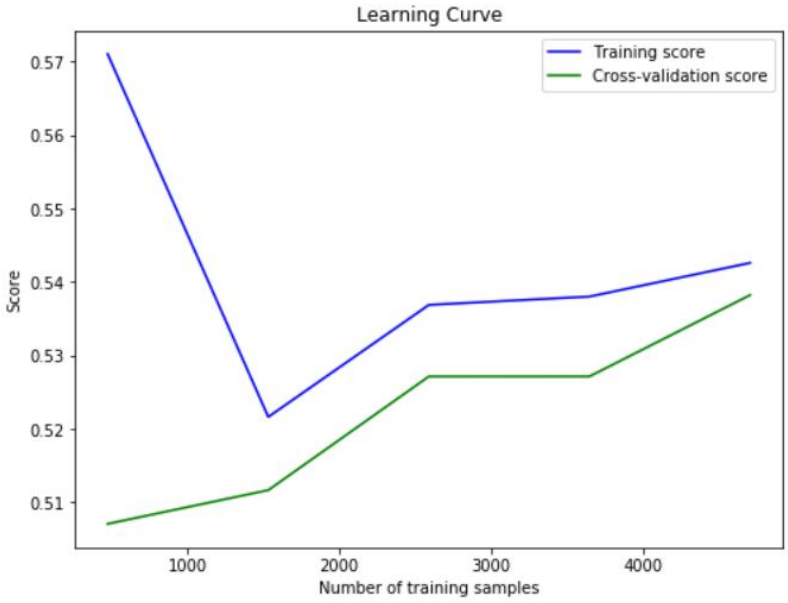
C represents the inverse of the regularization strength. The smaller this value, the stronger the regularization, which helps prevent overfitting. "np.logspace(-3, 2, 6)" generates 6 values between 10^{-3} and 10^{2}, which are evenly spaced on a logarithmic scale. The regularization type (penalty) considers three types: "L1", "L2", and "elasticnet". The first two types lead to sparse models (by setting some coefficients to zero to suppress unnecessary features) and penalize large coefficients, respectively, while the third type combines both "L1" and "L2". The optimization algorithm (solver) considers candidates such as the quasi-Newton method (lbfgs), coordinate descent (liblinear), SAGA, Newton's conjugate gradient method (newton-cg), and Stochastic Average Gradient (sag).

Model fitting:

```
# Model building using Logistic Regression
logreg_lo = LogisticRegression(max_iter=1000)

# Create the grid search object, set refit parameter to 'roc_auc', indicating ROC AUC as the refitting metric
grid_search_lo = GridSearchCV(estimator=logreg_lo, param_grid=param_grid_lo, cv=10,
                              scoring=scoring_lo, refit='roc_auc', n_jobs=-1, verbose=1)

# Fit the grid search object
grid_search_lo.fit(X_train_lo, y_train_lo)

# Output the best parameters
print("Best Hyperparameters:", grid_search_lo.best_params_)

# 3. Train the model with the best hyperparameters
best_logreg_lo = grid_search_lo.best_estimator_

# 4. Make predictions on the test set
y_pred_lo = best_logreg_lo.predict(X_test_lo)
```

Fitting result:

The optimal hyperparameter result is: {'C': 0.1, 'penalty': 'l1', 'solver': 'liblinear'}.



5.4.3 K-Nearest Neighbors(KNN)

KNN (Cover & Hart, 1968) is a type of algorithm that performs all calculations after classification, known as a lazy learning algorithm. For a data point to be predicted, the model finds the K nearest data points in the training set to that point, and then predicts the class of the point based on the classes of these K neighbors. The choice of K value can influence the model's decision-making results.

Hyperparameter selection:

```
# Grid search for hyperparameter tuning
param_grid_KNN = {
    'n_neighbors': np.arange(1, 31, 2),  # Odd numbers from 1 to 30 for a more detailed range
    'weights': ['uniform', 'distance'],
    'metric': ['minkowski', 'euclidean', 'manhattan', 'chebyshev'],
    'p': np.arange(1, 5)
}
```

In this context, “n_neighbors” represents the K value in the algorithm, and here, odd numbers from 1 to 30 are chosen to avoid ties during classification. If K is an even number, there may be a situation where the two classes have an equal number of samples, causing the vote to result in no clear majority class.

“Weights” indicates the contribution of each neighbor to the prediction result. “Uniform” means all neighbors have equal weight, while “distance” means that neighbors closer to the predicted point have a greater influence on the prediction result.

“Metric” refers to the distance metric used for calculating the distance, which can be the Euclidean distance (minkowski), Manhattan distance (manhattan), Chebyshev distance (chebyshev), or others. The parameter p is used in the Minkowski distance metric: when $p =$

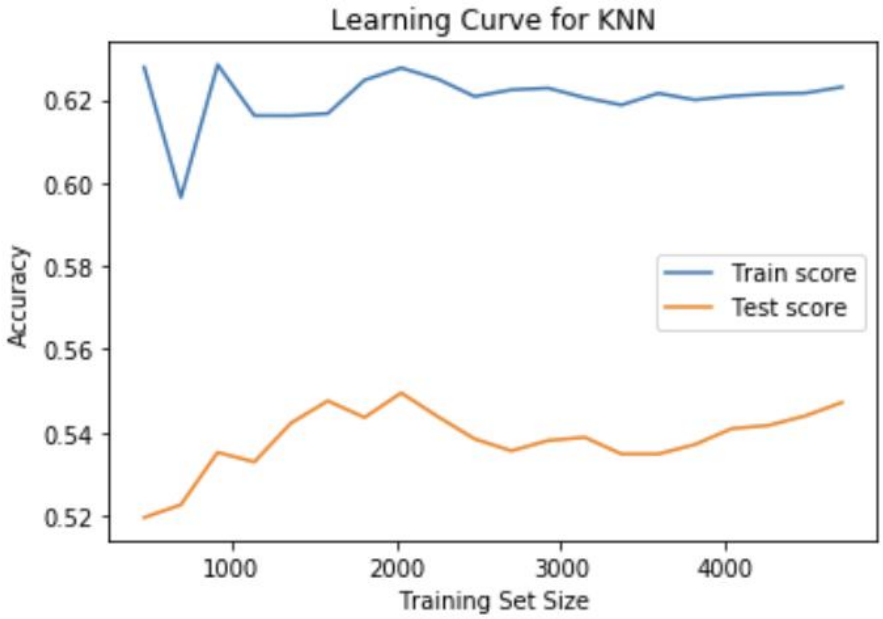
1, Minkowski distance becomes Manhattan distance; when $p = 2$, it becomes Euclidean distance; and when $p > 2$, the Minkowski distance forms a different type of metric.

Model fitting:

```
knn_KNN = KNeighborsClassifier()

# Perform hyperparameter tuning using GridSearchCV
grid_search_KNN = GridSearchCV(knn_KNN, param_grid_KNN, cv=10, n_jobs=-1, verbose=1)
grid_search_KNN.fit(X_train_KNN, y_train_KNN)

# Output the best parameters found
print("Best parameters found: ", grid_search_KNN.best_params_)

# Train the KNN model with the best parameters
best_knn_KNN = grid_search_KNN.best_estimator_

# Evaluate the model on the test set
y_pred_KNN = best_knn_KNN.predict(X_test_KNN)
```

Fitting result:

The optimal hyperparameter result is: {'metric': 'minkowski', 'n_neighbors': 21, 'p': 3, 'weights': 'uniform'}.



5.4.4 Random Forest Algorithm

Random Forest (Breiman, 2001) is an ensemble algorithm that performs classification or regression by constructing multiple decision trees and then making a final prediction by voting or averaging the results of the individual trees.

A decision tree is a supervised learning algorithm that derives decision rules from a set of training data with labels and represents these rules in a tree-like structure, solving classification and regression problems.

Random Forest uses bootstrap sampling (sampling with replacement) to randomly select multiple subsets from the training set, with each subset being used to train a separate decision tree. When splitting nodes, Random Forest does not use all the features; instead, it randomly selects a specific number of features to split the node, increasing the diversity between different trees. During the training of multiple decision trees, each tree uses a different training set and a subset of features. In classification problems, all decision trees produce a classification result, and Random Forest performs voting on these predictions, with the class having the most votes becoming the final predicted result.

Hyperparameter selection:

```
# Grid search for hyperparameter tuning
param_grid_RF = {
    'n_estimators': [100, 200, 300],  # Options for the number of trees
    'max_depth': [10, 20, None],  # Options for maximum depth, including no limit
    'min_samples_split': [2, 5, 10],  # Expanded options for minimum samples for node splitting
    'min_samples_leaf': [1, 2, 4],  # Increased options for minimum samples per leaf node
    'max_features': ['sqrt', 'log2'],  # Options for feature selection
    'bootstrap': [True, False],  # Whether to use bootstrapping or not
}
```

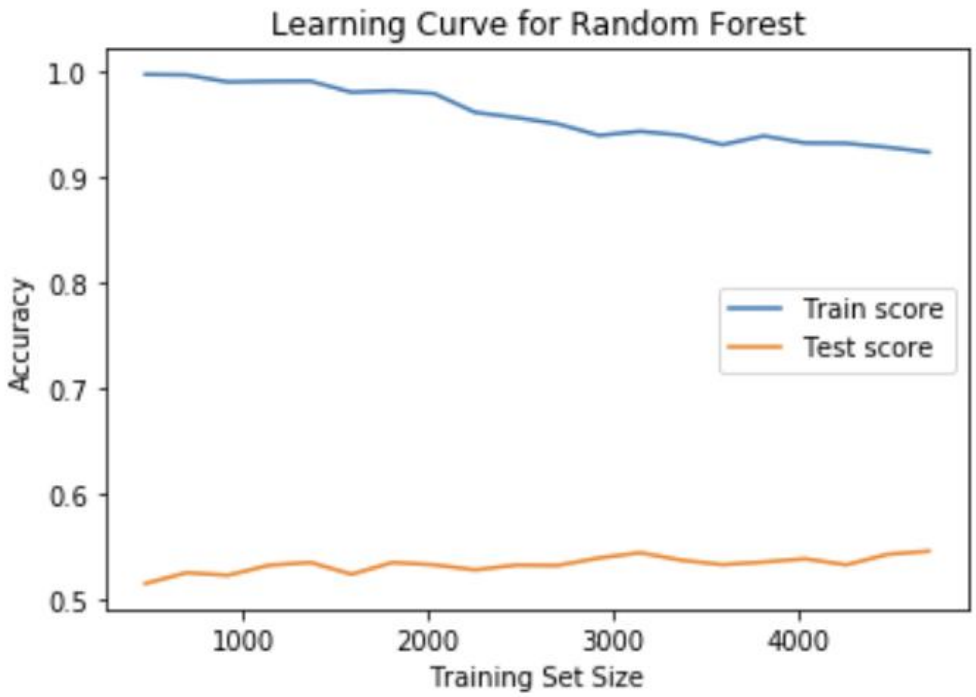
In this hyperparameter network, "n_estimators" refers to the number of decision trees. "Max_depth" refers to the maximum depth of the trees. "Min_samples_split" refers to the minimum number of samples required to split an internal node; increasing this value can reduce the complexity of the tree. "Min_samples_leaf" refers to the minimum number of samples required at a leaf node; increasing this value can make the model smoother. "Max_features" refers to the number of features randomly selected for each split, where 'sqrt' indicates that the number of features selected is the square root of the total number of features, and 'log2' means the number of features selected is the base-2 logarithm of the total number of features. Finally, "bootstrap" refers to whether bootstrap sampling is used.

Model fitting:

```
rf = RandomForestClassifier(random_state=42)

# Perform hyperparameter tuning using GridSearchCV
grid_search_RF = GridSearchCV(rf, param_grid_RF, cv=10, n_jobs=-1, verbose=1)
grid_search_RF.fit(X_train_RF, y_train_RF)

# Output the best parameters found
print("Best parameters found: ", grid_search_RF.best_params_)

# Train the Random Forest model with the best parameters
best_rf = grid_search_RF.best_estimator_

# Evaluate the model on the test set
y_pred_RF = best_rf.predict(X_test_RF)
```

Fitting result:

The optimal hyperparameter result is: {'bootstrap': False, 'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 10, 'n_estimators': 200}



5.4.5 Overall evaluation results

Machine learning models	Accuracy	Precision	Recall	F1 score
Support Vector Machine	0.563073	0.565719	0.700000	0.625737
Logistic Regression	0.568807	0.581276	0.620879	0.600425
K-nearest Neighbor Classification	0.529817	0.540835	0.654945	0.592445
Random Forest	0.567087	0.573460	0.664835	0.615776

5.5 Summary

This study found that Logistic Regression has the highest Accuracy and Precision, while the Support Vector Machine has the highest Recall and F1 score. This indicates that these two models have the best predictive performance when predicting on unseen data.

6 Quantitative Stock Selection Strategy

6.1 Data Preparation

Trading Signal Data

The trading signals are generated based on the prediction results of the machine learning models. The models predict the price changes of stocks for the upcoming week based on technical and financial factors, and output either a 0 or 1 label.

Label = 1: Indicates that the model predicts the stock will perform well in the future, generating a buy signal.

Label = 0: Indicates that the model predicts the stock will perform poorly in the future, generating a sell signal.

The trading labels used in this study are derived from the predictions made. These labels represent weekly data from November 8, 2024, to November 29, 2024. Below is an example of the trading label data ("trade_data") for the SVM model.

Trading Data

The weekly actual trading data of the selected stocks is imported from Wind, providing accurate data ("df_pred") for subsequent trade backtesting and for verifying the accuracy of the original data.

ts_code	20241108	20241115	20241122	20241129
000009.SZ	0	1	1	1
000021.SZ	1	0	0	1
000027.SZ	0	0	1	0
000031.SZ	1	1	0	0
000032.SZ	0	0	1	0
...	...	...	...	...
688777.SH	0	1	1	1
688778.SH	0	1	0	1
688779.SH	0	1	0	0
688819.SH	1	0	1	0
689009.SH	0	1	1	1

436 rows × 4 columns

ts_code	20241108	20241115	20241122	20241129
000009.SZ	10.48	9.84	9.35	9.86
000021.SZ	21.28	20.35	19.72	20.16
000027.SZ	6.86	6.62	6.52	6.62
000031.SZ	3.41	3.10	3.05	3.29
000032.SZ	20.13	19.53	18.88	18.66
...	...	...	...	...
688777.SH	51.09	49.28	47.09	48.46
688778.SH	55.88	50.44	47.86	49.48
688779.SH	6.34	6.14	6.07	6.05
688819.SH	29.97	29.90	28.53	28.90
689009.SH	47.01	47.54	43.06	44.50

436 rows × 4 columns

6.2 Strategy Design

In this section, the four models trained in the previous section will be used to construct a quantitative trading strategy, followed by backtesting. Given the timeliness of the information, the machine learning models are retrained weekly during the backtest, and predictions are made using a one-step-ahead forecasting approach. The buy/sell strategy and key parameters for the constructed quantitative strategy are outlined below.

Initial Capital: 10,000.

Rebalancing Frequency: Weekly.

Backtest Period: November 2, 2024, to November 29, 2024.

Benchmark Return: CSI 500 constituent stocks.

Selection of Trading Assets: The selected stocks are the CSI 500 constituent stocks after removing specific stocks.

Number of Shares Bought per Transaction: 10 shares.

Position Management: The quantity and cost price of the held stocks are recorded in the "positions" dictionary for tracking and management.

Trade Execution: Go through each trading day and execute buy or sell operations according to the model's labels and current stock prices. When buying, deduct the corresponding stock purchase costs from the cash and record the position information. When selling, add the proceeds from the sale back to the cash and remove the corresponding stock from the position.

Code Presentation:

When the trading label in the "df_pred" table is 1 for a particular stock, it indicates a positive outlook for the stock's future performance, hence the stock with a label of 1 will be bought. At the next time point, the holdings will be adjusted according to the latest trading signals: if a stock's label changes from 1 to 0, the stock will be sold; if a stock's label changes from 0 to 1, the stock will be bought; if the stock's label remains 1, the current holding will be maintained.

```
def trading_strategy(df_pred, trade_data, stock_selected_code, specific_dates, initial_cash=100000):
    cash = initial_cash
    positions = {}  # Store the stock positions and cost prices
    portfolio_value = initial_cash

    # Iterate through each trading day
    for date in specific_dates:
        # Iterate through each stock
        for ts_code in stock_selected_code:  # Skip the date column
            signal = df_pred.loc[ts_code, date]
            current_price = trade_data.loc[ts_code, date]

            if signal == 1:
                if ts_code not in positions:
                    # Buy the stock
                    if cash > 0:
                        shares = 10  # Assume buying 10 shares each time
                        cash -= shares * current_price
                        positions[ts_code] = {'shares': shares, 'cost': current_price}
                        print(date, ': Buy', shares, ts_code, '\n', cash)

                else:
                    # Hold the position
                    pass
```

```
            elif signal == 0 and ts_code in positions:
                # Sell the stock
                cash += positions[ts_code]['shares'] * current_price
                print(date, ': Sell', positions[ts_code]['shares'], ts_code, '\n', cash)
                del positions[ts_code]

        # Update the portfolio value
        portfolio_value = cash
        for ts_code, info in positions.items():
            portfolio_value += info['shares'] * current_price

    # Print the final portfolio value
    print(f'Final Portfolio Value: {portfolio_value}')
    return portfolio_value
```

6.3 Backtest Results and Analysis

6.3.1 Backtest Results

To evaluate the effectiveness of the quantitative stock selection strategy, we used backtesting to apply the model's predicted labels to past data, simulating the actual trading process and calculating the returns during the backtest period. During the backtest, we compared the strategy's performance with the returns of the CSI 500 Index. The table is as follows:

Machine learning models	Accuracy	Precision	Recall	F1 score	Return (%)
Support Vector Machine	0.563073	0.565719	0.700000	0.625737	60.817800
Logistic Regression	0.568807	0.581276	0.620879	0.600425	51.258200
K-nearest Neighbor Classification	0.529817	0.540835	0.654945	0.592445	44.253600
Random Forest	0.567087	0.573460	0.664835	0.615776	52.064300
CSI 500 index					-0.343763

6.3.2 Comprehensive Evaluation

Compared to the CSI 500 index

all machine learning models perform better, indicating that machine learning models have certain advantages in quantitative trading strategies.

Comparison between models

Based on these indicators, if the goal is to maximize returns, SVM may be the best choice. If more attention is paid to predictive accuracy, Logistic Regression may be more appropriate.

The SVM displayed the best return, achieving 60.82%, far surpassing other models, indicating that this model performs excellently in stock prediction.

Logistic Regression and Random Forest also showed strong returns, with 51.26% and 52.06% respectively, both significantly outperforming the CSI 500 Index. Although Random Forest slightly outperformed Logistic Regression in terms of returns, the difference was minimal.

The KNN model achieved a return of 44.25%, which, although lower than the other machine learning algorithms, still outperformed the CSI 500 Index, indicating a certain level of predictive capability.

Meanwhile, the CSI 500 Index had a return of -0.34%, showing poor performance, which means that a strategy that directly tracks the index did not generate positive returns during the backtest period.

From the backtest results, machine learning models clearly outperformed traditional index tracking strategies, especially the SVM and Random Forest, which excelled in predicting stock price trends. Investors may consider prioritizing these models to construct quantitative strategies.

7 Summary

This study constructs a quantitative investment strategy using machine learning algorithms through research on quantitative investment and machine learning. It reveals the feasibility and significance of dimensionality reduction and the effectiveness of machine learning in quantitative investment.

First, the study analyzes data from the constituents of the CSI 500 index from August 16, 2024, to November 29, 2024. Principal Component Analysis (PCA) is used to reduce the dimensionality of selected financial and technical indicators, successfully reducing the data to eight dimensions (with a cumulative explained variance of over 85%). Next, four machine learning models—SVM, Logistic Regression, KNN, and Random Forest—are applied to predict the direction of stock price movement for the following week. The models' performance is evaluated using accuracy, precision, recall, and F1 score. The models are ranked in descending order of predictive performance as follows: SVM, Random Forest, Logistic Regression, and KNN. Finally, the trained machine learning models are used to construct a quantitative stock selection strategy, and the returns of each model under the same strategy are compared. The results show that the SVM model yields the highest returns. Compared with market benchmarks, the quantitative strategy built using machine learning

significantly outperforms the market index, demonstrating the effectiveness and potential of machine learning in quantitative investment.

8 Limitations and Outlook

Factor Selection

The current model's factor selection has room for improvement. More financial and technical indicators can be added to more comprehensively assess the performance of stocks.

Model Selection

The current model selection has certain limitations, mainly in terms of the variety of models used and the lack of complex models. The models used in this study, such as Support Vector Machines (SVM), Random Forest, Logistic Regression, and K-Nearest Neighbors (KNN), although effective in stock prediction, have limitations in handling nonlinear relationships and high-dimensional data. More complex models, such as XGBoost, Deep Neural Networks (DNN), and Long Short-Term Memory Networks (LSTM), have stronger data processing capabilities and predictive accuracy but were not fully utilized in this study due to time and resource constraints. Future research could consider incorporating these advanced models to improve predictive performance.

Parameter Optimization

The model uses grid search for parameter tuning, which has limitations and does not guarantee the optimal parameters. Furthermore, due to time constraints, not all possibilities were considered during the tuning process.

Strategy Construction

In constructing the trading strategy, the limited data availability has resulted in a short time span for the model. If sufficient time was available, we could consider the timeliness of the information. Assuming more than five years of data, we could retrain the machine learning model every three months during backtesting and use a one-step-ahead prediction approach, allowing the quantitative strategy to be continuously updated rather than relying solely on fixed model parameters.

References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5-32. https://doi.org/10.1023/A:1010933404324

Cortes, C., & Vapnik, V. N. (1995). Support vector networks. *Machine Learning, 20*, 273-297.

Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, 13*, 21-27.

Chen, J. H., & Luo, Y. H. (2014). A multi-factor stock selection model based on pure technical indicators. *Zhao Shang Securities Research Report.*

Ding, P. (2014). *Quantitative investment: Strategies and techniques* (pp. 24-29). Beijing: Electronics Industry Press.

Li, B., Shao, X. Y., & Li, Y. Y. (2019). Research on machine learning-driven fundamental quantitative investment. *China Industrial Economics, (08)*, 61-79.

Sharpe, W. F. (1964). Capital asset prices: A theory of market equilibrium under conditions of risk. *Journal of Finance, 19*(3), 425-442.

Uçkan, T. (2024). Integrating PCA with deep learning models for stock market forecasting: An analysis of Turkish stock markets. *Journal of King Saud University - Computer and Information Sciences, 36*(8). https://doi.org/10.1016/j.jksuci.2024.102162

Zhou, Z.-H. (2016). *Machine learning* (pp. 121-137). Tsinghua University Press.

Sharpe, W. F. (1964). Capital asset prices: A theory of market equilibrium under conditions of risk. *Journal of Finance, 19*(3), 425-442.